OpenAI-Only RAG Amazon Extraction Plan

Summary

- Replace the current schema-first design with a retrieval-first pipeline driven by `asin + required_fields`.
- Scrape the full Amazon PDP once, normalize all visible product content into chunked documents, retrieve relevant chunks using OpenAI embeddings only, then extract final field values using an OpenAI chat model.
- Remove schema files from the main architecture. `product_type` becomes optional metadata only.
- Use only the OpenAI API for semantic retrieval and final extraction; no Ollama, no local inference runtime, and no BM25 fallback.
- Keep classic RAG as the default path, with a lightweight agentic fallback only for weak, conflicting, or low-confidence field retrieval.

Key Changes

- Input contract:
  - Required: `asin`, `required_fields`
  - Optional: `product_type`
  - `required_fields` accepts comma/pipe/semicolon-separated field names per row.
- Content acquisition:
  - Keep Selenium-based PDP loading on `amazon.com` with New York, USA delivery context.
  - Capture title, bullets, facts/details tables, description/A+ text, selected variations, and other visible product text.
  - Normalize the page into semantic chunk records with metadata: `asin`, `section`, `text`, `dom_hint`, `order`, and source priority.
  - Chunk semantically by block rather than arbitrary token windows: title, individual bullets, individual facts/details rows, selected variation labels/values, and coherent description/A+ paragraphs.
- Retrieval pipeline:
  - Build dense retrieval over normalized chunks using OpenAI embeddings only.
  - No BM25 or keyword retriever is part of the plan.
  - Embed both chunk text and requested-field queries with the configured OpenAI embedding model.
  - Build requested-field queries using a hybrid strategy:
    - first use a maintained field-query template map for known fields
    - if no template exists, fall back to dynamic query text derived from the raw field name
    - if dynamic fallback is still too weak for retrieval quality, optionally call an OpenAI chat model to generate an expanded retrieval query using synonyms, expected PDP phrasing, and likely sec-

tion terminology
  - Rank chunks primarily by cosine similarity and return top-K evidence chunks per requested field.
  - Add light section-aware prioritization where useful; for structured or binary attributes, prefer facts/details and selected variation text over descriptive marketing copy.
  - Add a lightweight agentic fallback only for failed, weak, conflicting, or low-confidence fields:
    1. retry retrieval with an expanded field query using field-specific synonyms and expected PDP phrasing
    2. if still weak, retry with section-aware targeting such as facts/details, title, and selected variations
    3. if still weak, retry with a broader evidence set such as top 8 chunks plus title and details rows
    4. only then allow final blank output
  - Support a broader-evidence fallback for low-confidence fields: first pass with top 3-5 chunks, then retry with a larger evidence set such as top 8 chunks plus title and details rows before blanking.
  - Add local caching for embeddings keyed by normalized chunk/query text to reduce repeated API cost.
  - Log retrieval diagnostics for low-confidence or unsupported outputs, including query text, top retrieved chunks, retry path taken, and similarity scores.
- Extraction pipeline:
  - For each requested field, call an OpenAI chat/completions model with:
    * requested field name
    * product title
    * top retrieved chunks
    * strict response contract requiring JSON with `value`, `evidence`, `source`, and `confidence`
  - Run deterministic post-processing after model output for whitespace cleanup, numeric/unit normalization, yes/no normalization, and blank/null handling.
  - Add deterministic post-retrieval resolvers for simple highly structured fields, especially binary yes/no attributes, before final blanking. Example: normalize explicit evidence such as `caffeinated`, `contains caffeine`, `decaf`, or `decaffeinated` into `Yes` or `No`, preferring selected variation and facts/details over generic description text.
  - Treat model confidence as a proposed score that must be validated using evidence explicitness, source quality, chunk agreement, and ambiguity or conflict checks.
  - Reject low-confidence or unsupported outputs and write blank values instead of guessed values only after the retrieval fallback, expanded-query retry, and deterministic resolver paths are exhausted.
- Output contract:

  - One CSV per run.
  - Core columns: `asin`, `product_type`, `status`, `url`, `scraped_at`, `title`, `error`.
  - For each requested field, output `<field>`, `<field>_source`, `<field>_confidence`, and `<field>_evidence`.
- Deprecations:
  - Remove schema YAML files from the planned extraction path.
  - Existing deterministic parsers remain only as post-processing helpers, not as the primary extraction strategy.

Public Interfaces

- `ScrapeJob` carries `asin`, optional `product_type`, and `requested_fields`.
- Add a document-store layer that returns normalized chunk records for one PDP.
- Add an OpenAI embedding client for chunk/query embedding and similarity scoring.
- Add an OpenAI extraction client for structured field extraction from retrieved evidence.
- Add a field-query strategy layer that supports:
  - maintained query templates
  - dynamic fallback from raw field names
  - optional OpenAI-generated expanded retrieval queries when template coverage is missing or first-pass retrieval is weak
- Add retrieval diagnostics output for debugging low-confidence or blank results, including whether lightweight agentic fallback was triggered.
- Require configuration through environment variables:
  - `OPENAI_API_KEY`
  - `OPENAI_EMBEDDING_MODEL`
  - `OPENAI_CHAT_MODEL`
- Fail fast at startup if any required OpenAI configuration is missing.

Test Plan

- Unit tests:
  - semantic chunk normalization from title, bullets, facts/details rows, description paragraphs, and variations
  - dense retrieval ranking from mocked embeddings
  - field-query template mapping with dynamic fallback
  - OpenAI-generated expanded query fallback behavior when template coverage is missing or retrieval is weak
  - embedding-cache keying and reuse
  - extraction-response parsing and post-processing
  - deterministic resolver behavior for simple yes/no fields
  - low-confidence fallback handling before final blanking

  - lightweight agentic retry ordering: expanded query -> section-aware retry -> broader evidence
- Fixture tests:
  - one PDP where the answer is in facts/details
  - one PDP where the answer is only in title/bullets
  - one PDP where the requested field appears multiple times across sections
  - one PDP where the requested field is absent
  - one PDP where the raw field name is weak but expanded query retrieval succeeds
- Acceptance scenarios:
  - input with only `asin + required_fields` runs successfully without schema files
  - `caffeine_per_serving` resolves from retrieved title/description evidence
  - `roast_level` resolves from retrieved title/bullet evidence
  - repeated field mentions on a PDP do not cause incorrect extraction when one source is more explicit or authoritative than others
  - binary fields such as `caffeinated` normalize to `Yes` or `No` from explicit PDP evidence instead of blanking when the answer is present
  - missing field-query templates do not block retrieval because dynamic query fallback is available
  - weak first-pass retrieval can be improved by expanded field-query retry before broader evidence fallback
  - repeated runs reuse cached embeddings for unchanged content
  - unrelated high-similarity text does not get written as the field value
  - low-confidence first-pass retrieval triggers lightweight agentic fallback before export
  - low-confidence outputs are exported as blank with evidence/confidence metadata preserved only after fallback and deterministic resolution fail
  - retrieval diagnostics make blank outputs explainable and debuggable

Assumptions

- OpenAI API access is available and acceptable for both embeddings and extraction.
- No local LLM runtime, Ollama integration, BM25 retriever, or schema-driven fallback is in scope.
- Retrieval is dense-only and per-run, with local embedding cache persisted between runs.
- Lightweight retry logic for failed fields is in scope, including expanded-query retry and optional OpenAI-generated retrieval-query fallback, but full agentic RAG is not the default architecture.
- The system is designed for dynamic requested fields, but extraction quality still depends on the requested field being inferable from PDP evidence.